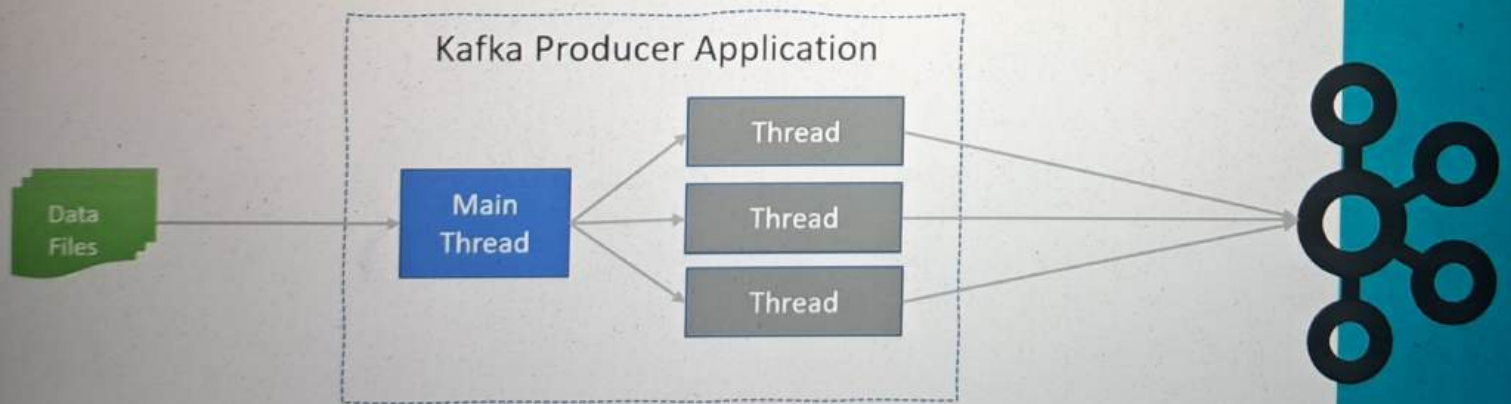
Creating Multi-threaded Producer?

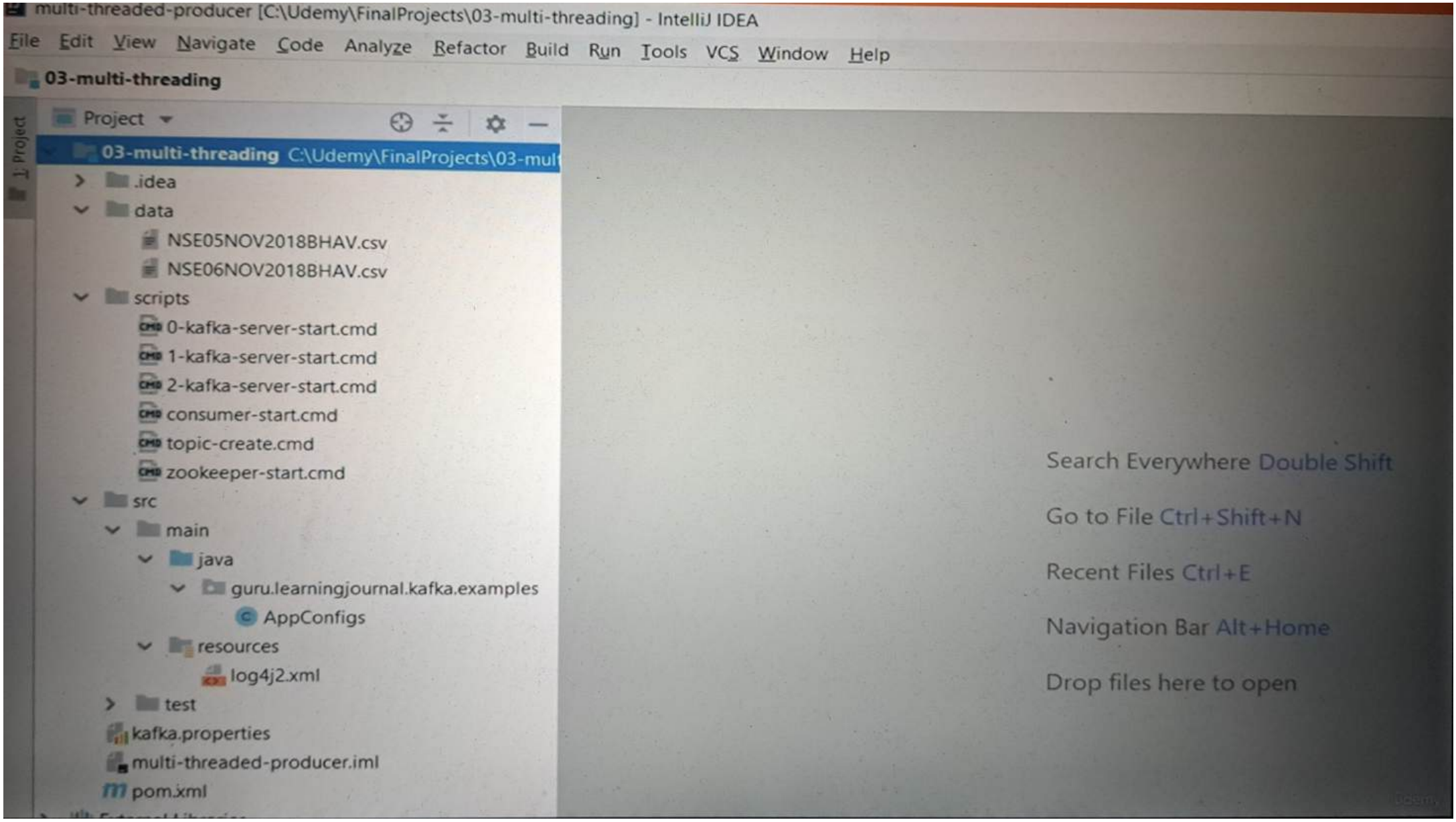
Scaling Kafka Producer

Problem Statement

Create a multi-threaded Kafka Producer that sends data from a list of files to a Kafka topic such that independent thread streams each file.



LEARNING JOURNAL


multi-threaded-producer [C:\Udemy\FinalProjects\03-multi-threading] - IntelliJ IDEA
File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help
03-multi-threading
Project
03-multi-threading C:\Udemy\FinalProjects\03-mult
.idea
data
NSE05NOV2018BHAV.csv
NSE06NOV2018BHAV.csv
scripts
0-kafka-server-start.cmd
1-kafka-server-start.cmd
2-kafka-server-start.cmd
consumer-start.cmd
topic-create.cmd
zookeeper-start.cmd
src
main
java
guru.learningjournal.kafka.examples
AppConfigs
resources
log4j2.xml
test
kafka.properties
multi-threaded-producer.iml
pom.xml
Search Everywhere Double Shift
Go to File Ctrl+Shift+N
Recent Files Ctrl+E
Navigation Bar Alt+Home
Drop files here to open

```java
package guru.learningjournal.kafka.examples;

class AppConfigs {
    final static String applicationID = "Multi-Threaded-Producer";
    final static String topicName = "nse-eod-topic";
    final static String kafkaConfigFileLocation = "kafka.properties";
    final static String[] eventFiles = {"data/NSE05NOV2018BHAV.csv","data/NSE06NOV2018BHAV.csv"};
}
```

multi-threaded producer | log4j2.xml | AppConfigs.java | kafka.properties

```
1  bootstrap.servers=localhost:9092,localhost:9093
```

Dispatcher.java | DispatcherDemo.java

```java
import java.util.Scanner;

public class Dispatcher implements Runnable {
    private static final Logger logger = LogManager.getLogger();
    private String fileLocation;
    private String topicName;
    private KafkaProducer<Integer, String> producer;

    Dispatcher(KafkaProducer<Integer, String> producer, String topicName, String fileLocation) {
        this.producer = producer;
        this.topicName = topicName;
        this.fileLocation = fileLocation;
    }

    @Override
    public void run() {
        logger.info( s: "Start Processing " + fileLocation);
        File file = new File(fileLocation);
        int counter = 0;

        try (Scanner scanner = new Scanner(file)) {
            while (scanner.hasNextLine()) {
                String line = scanner.nextLine();
                producer.send(new ProducerRecord<>(topicName,  key: null, line));
                counter++;
            }
            logger.info( s: "Finished Sending " + counter + " messages from " + fileLocation);
        } catch (FileNotFoundException e) {
            throw new RuntimeException(e);
        }

    }
}
```

Dispatcher › run()

Event Log

Dispatcher.java | DispatcherDemo.java

```
        Properties props = new Properties();
        try{
            InputStream inputStream = new FileInputStream(AppConfigs.kafkaConfigFileLocation);
            props.load(inputStream);
            props.put(ProducerConfig.CLIENT_ID_CONFIG,AppConfigs.applicationID);
            props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, IntegerSerializer.class);
            props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
        }catch (IOException e){
            throw new RuntimeException(e);
        }

        KafkaProducer<Integer,String> producer = new KafkaProducer<Integer, String>(props);
        Thread[] dispatchers = new Thread[AppConfigs.eventFiles.length];
        logger.info( s: "Starting Dispatcher threads...");
        for(int i=0;i<AppConfigs.eventFiles.length;i++){
            dispatchers[i]=new Thread(new Dispatcher(producer,AppConfigs.topicName,AppConfigs.eventFiles[i]));
            dispatchers[i].start();
        }

        try {
            for (Thread t : dispatchers) t.join();
        }catch (InterruptedException e){
            logger.error( s: "Main Thread Interrupted");
        }finally {
            producer.close();
            logger.info( s: "Finished Dispatcher Demo");
        }
    }
}
```

DispatcherDemo

Event Log